PacketArch

Installation Guide

OT Traffic Simulation Platform — deployment options, system requirements, and setup commands

Release	v1.22.0
Build commit	unknown
Document date	October 22, 2026
Repository	github.com/ip-aegis/PacketArch
License	GPL-3.0

Choosing an Installation Method

PacketArch ships as a Docker Compose stack (FastAPI backend, React/Nginx frontend, PostgreSQL/TimescaleDB, and Redis). There are four supported ways to install it depending on whether your environment has internet access and how you prefer to manage upgrades. All four land on the same first-run setup wizard.

Method	Best for	Internet?	Upgrades via
1. Git-clone production install	A standard internet-connected Linux server	Required	In-app button or git pull + rebuild
2. Offline / air-gapped bundle	Isolated labs with no internet egress	Not required	New release tarball (install.sh --upgrade)
3. Virtual appliance (OVA)	Drop-in VM for VirtualBox / VMware / ESXi	First boot only	In-app upgrade button
4. Developer / source setup	Local development & contributing	Required	git pull

Variants. Each release is built in two flavors. The **full** variant includes live traffic agents and the live deployment dashboard. The **PCAP-only** variant (`LIVE_TRAFFIC_ENABLED=false`) ships as an AI-assisted PCAP generator with the live-agent half disabled.

System Requirements

Resource	Minimum	Recommended
Operating system	Linux x86_64 (Ubuntu 22.04+, RHEL 9, Fedora)	Ubuntu 22.04 / 24.04 LTS
CPU	2 cores	4+ cores
Memory	8 GB RAM	16 GB RAM
Disk	20 GB free	40 GB+ (images + DB + PCAP output)
Docker	24.0+ with Compose plugin	Latest stable Docker CE
Privileges	sudo (write /opt, bind 443)	—

Required network ports

Port	Service	Exposure
443	Web UI (HTTPS, self-signed by default)	Inbound: admin -> server
443	Agent <-> server WebSocket (wss)	Inbound: agent -> server
5432	PostgreSQL / TimescaleDB	Localhost only
6379	Redis	Localhost only

Optional egress. PacketArch runs fully offline. To enable AI-powered scenario generation, the backend needs outbound HTTPS to `api.anthropic.com`. Leave `AI_ENABLED=false` to disable it; AI surfaces hide automatically.

Method 1 — Git-Clone Production Install

The standard path for an internet-connected server. It installs Docker if missing, clones the repository, generates secrets, and builds and starts the stack. This install is **self-upgradeable** from the UI (Settings -> System).

One-line bootstrap

```
curl -sSL https://raw.githubusercontent.com/ip-aegis/PacketArch/master/scripts/server-init.sh \
  | bash
```

Run as a regular user with sudo privileges (**not** root). The script performs five steps:

- **1/5** Installs Docker CE + Compose plugin if not present.
- **2/5** Installs Git if not present.
- **3/5** Clones `ip-aegis/PacketArch` to `~/packetarch` (branch `master`).
- **4/5** Generates `.env` with random `POSTGRES_PASSWORD`, `SECRET_KEY`, `ENCRYPTION_KEY`, the host `DOCKER_GID`, and self-upgrade pointers (`HOST_INSTALL_DIR`, `COMPOSE_PROJECT_NAME=packetarch`).
- **5/5** Checks Docker egress (`check-docker-egress.sh`), runs `docker compose up -d --build`, and then *the script* prints the access URLs. The compose command itself prints no URLs — if you follow the manual equivalent below, use the table in “Access and first login”.

Manual equivalent

```
git clone https://github.com/ip-aegis/PacketArch.git ~/packetarch
cd ~/packetarch
# create .env (see Environment Variables section), then:
sudo ./scripts/check-docker-egress.sh      # preflight; see below
sudo docker compose up -d --build
```

Access and first login

`docker compose up -d --build` prints no URLs of its own. These are the addresses either path produces:

Service	URL
Frontend (the platform)	`https://<server-ip>/`
Health probe	`https://<server-ip>/health`
API docs (Swagger)	`https://<server-ip>/api/docs — only when DEBUG=true`
pgAdmin (tools profile)	`http://localhost:5050 via SSH tunnel`

The certificate is self-signed by default, so expect a browser trust warning. If the page loads but login reports **Backend unreachable**, nginx cannot reach the application — see “Troubleshooting”.

After it finishes

- Open `https://<server-ip>/` and accept the self-signed certificate.

- API docs (`/api/docs`) are served only when `DEBUG=true`. A production install sets `DEBUG=false`, so they are disabled on purpose — do not enable DEBUG on a production box just to read them (it also turns on SQL echo and returns raw errors to clients).
- Complete the first-run setup wizard to create the admin account (see the dedicated section).
- Open inbound `443` (and `80`) in your cloud/network firewall for off-box access.

> Docker group changes from a fresh Docker install take effect on next login. If `docker` commands need sudo right after install, log out and back in.

Method 2 — Offline / Air-Gapped Bundle

A self-contained tarball with every Docker image pre-saved, so no registry or internet access is needed at install time. This is the right choice for isolated labs.

Building the bundle (on a connected machine)

```
./scripts/build-release.sh               # full variant
PCAP_ONLY=1 ./scripts/build-release.sh    # PCAP-only variant
# output: dist/packetarch-1.22.0-offline.tar.gz
```

The bundle contains all images (backend, frontend, agent, postgres/timescaledb, redis) saved to tarballs, an offline `docker-compose.yml`, `install.sh`, `README_SITE.md`, and the license files. Tagging `v*` in CI builds both variants automatically and attaches them to the GitHub Release.

Installing on the target server

```
tar xzf packetarch-*-offline.tar.gz
cd packetarch-*-offline
sudo ./install.sh
```

The installer (idempotent, safe to re-run):

- **[1/5]** Stages files into `/opt/packetarch` (override with `--install-dir PATH`).
- **[2/5]** `docker loads` every bundled image into the local daemon.
- **[3/5]** Generates `.env` with random secrets (chmod 600). **No admin password is generated** — you pick it in the wizard.
- **[4/5]** Starts the stack with `docker compose --env-file .env up -d`.
- **[5/5]** Waits up to 5 minutes for the backend healthcheck.

install.sh flags

Flag	Effect
`--upgrade`	Load new images and restart; preserve existing .env + volumes. Requires an existing .env.
`--force-env`	Overwrite an existing .env (generates new secrets — breaks existing logins and DB access).
`--install-dir PATH`	Where to place the installation (default /opt/packetarch).

Prerequisites checked by the installer

- Must run as root (use `sudo`).
- Docker installed and the Docker Compose plugin available.
- Run from inside the extracted release directory (a `VERSION` file must be present).

Method 3 — Virtual Appliance (OVA)

A distributable `.ova` you import into VirtualBox, VMware Workstation/Player, or ESXi/vSphere. The appliance is a real git clone pinned to a release tag using the production compose file, so it stays self-upgradeable from the UI. The release pipeline now builds the `.ova` in CI and attaches it to the GitHub Release, so most operators download it rather than building it — you only rebuild at major releases.

Building the OVA

```
# Fedora:  sudo dnf install guestfs-tools qemu-img git curl
# Ubuntu:  sudo apt-get install libguestfs-tools qemu-utils git curl

sudo ./scripts/ova/build-ova.sh
# output: dist/packetarch-1.22.0-appliance.ova
```

Useful build overrides:

Variable	Default	Purpose
`OVA_GIT_REF`	latest v* tag	Tag/branch the appliance is pinned to
`DISK_SIZE`	60G	Thin-provisioned virtual disk size
`VM_CPUS / VM_MEM`	4 / 8192	OVF-suggested resources
`CONSOLE_PASS`	packetarch	Console password for the ubuntu user

Deploying the appliance

- Import the `.ova` into your hypervisor.
- Place it on a network with **DHCP and internet**, then power on.
- **First boot builds from source (~10–15 min)**; later boots start in seconds. Watch `/var/log/packetarch-firstboot.log`.
- Browse to `https://<appliance-ip>/`, accept the cert, and complete the wizard.

> Change the default console login (`ubuntu` / `packetarch`) after first login. For a truly air-gapped, no-build appliance, use the offline tarball (Method 2) on a plain VM instead.

Method 4 — Developer / Source Setup

For local development and contributing. Runs the backend and frontend natively with only PostgreSQL and Redis in Docker.

Prerequisites

- Docker & Docker Compose
- Python 3.11+ with Poetry
- Node.js 18+ with pnpm

Setup

```
git clone https://github.com/ip-aegis/PacketArch.git
cd PacketArch

# 1. Dev .env (password must match the backend default DATABASE_URL)
printf 'POSTGRES_PASSWORD=packetarch_dev\nSECRET_KEY=dev-secret-not-for-production\n' > .env

# 2. Database + Redis (from the repo root)
docker compose up -d postgres redis

# 3. Backend (http://localhost:8001)
cd backend && poetry install
poetry run uvicorn app.main:app --reload --host 0.0.0.0 --port 8001

# 4. Frontend (new terminal -> http://localhost:3001)
cd frontend && pnpm install && pnpm dev
```

Development ports

Service	Port
Backend (FastAPI)	`8001`
Frontend (Vite)	`3001`
PostgreSQL	`5432`
Redis	`6379`
pgAdmin (optional)	`5050`

On Windows, use `python -m poetry run uvicorn ...` if Poetry is not on PATH.

First-Run Setup Wizard

Regardless of install method, a fresh install lands on a setup wizard at `https://<server>/` instead of a login page. Until the wizard finishes, every API route except setup/about/health returns 503. The wizard walks through four steps:

- **Admin account** — username, password, optional email.
- **Site identity** — site name, server FQDN/IP (used in agent install commands), time zone.
- **Capabilities** — optional AI (Anthropic API key) and optional Cisco Cyber Vision import; both can be added later under Settings.
- **Confirm** — review, accept the GPL-3.0 license, click **Complete setup**. You are auto-logged-in to the dashboard.

> **The setup wizard is unprotected** — the first person who reaches the URL becomes the admin. Complete it **before** anyone else can browse to the server. Save the admin password somewhere safe; it is the only credential to recover it from.

Recovering from a botched setup

If someone else claimed admin during the window, or you want to start over, reset and re-run the wizard:

```
cd /opt/packetarch
sudo docker compose exec postgres psql -U packetarch -d packetarch -c \
  "DELETE FROM users; UPDATE system_settings SET value='false' WHERE key='setup.completed';"
sudo docker compose restart backend
```

Upgrades from pre-wizard installs do not show the wizard: on every boot, auto-graduation flips setup as complete if an admin user already exists.

Environment Variables (.env)

Generated automatically by the installers. Treat `.env` as secret material (chmod 600).

Variable	Description
`POSTGRES_PASSWORD`	Database password (random per install).
`SECRET_KEY`	JWT signing key (random per install).
`ENCRYPTION_KEY`	Fernet key that encrypts stored secrets (CV token, AI keys) at rest. Persists them across restarts.
`ADMIN_PASSWORD`	Left blank on fresh installs (wizard creates admin). A set value is a legacy headless-bootstrap path.
`AI_ENABLED`	Gates AI features. Default true (git install) / false (offline bundle).
`LIVE_TRAFFIC_ENABLED`	Gates live agents + deployment dashboard. false in the PCAP-only variant.
`DOCKER_GID`	Host docker group id so the backend can use the Docker socket.
`HOST_INSTALL_DIR / COMPOSE_PROJECT_NAME`	Targets for the in-app self-upgrade (git-clone installs).
`BUILD_COMMIT / BUILD_DATE`	Build provenance stamped at release-build time; surfaced in the UI footer / About page.
`COMPOSE_SUBNET`	Subnet for the stack's own bridge network, PINNED rather than taken from Docker's pools (default 10.200.0.0/24). A site or VPN route that overlaps Docker's pools makes the stack come up and still be unreachable — see Troubleshooting. Changing it needs the network recreated (down, then up).
`DOCKER_BUILD_NETWORK`	Leave unset on a healthy host. 'host' makes every image build use the host network stack — a stopgap for a host whose bridged build sandbox has no egress.
`DEBUG`	false in production.

Post-Install Tasks

Provide your own TLS certificate

By default the frontend mints a self-signed cert. To use a real one, drop it in `./certs` and restart:

```
sudo mkdir -p /opt/packetarch/certs
sudo cp server.crt /opt/packetarch/certs/server.crt
sudo cp server.key /opt/packetarch/certs/server.key
sudo chmod 600 /opt/packetarch/certs/server.key
sudo docker compose restart frontend
```

Install remote traffic agents (full variant)

Agents connect outbound over WebSocket (TLS on 443) — no inbound ports needed on the agent host. Generate a token in the UI under Settings -> Agents, then on each agent box:

```
curl -fsSLk https://<your-server>/agent/install.sh | sudo bash -s -- \
    --server https://<your-server> --token <agent-token> --insecure
```

Turn AI on or off

Edit `.env`, set `AI_ENABLED=true|false`, then `docker compose up -d backend`. When on, set an Anthropic API key under Settings -> AI Provider (keys are encrypted at rest).

Upgrades

There are three upgrade mechanisms:

- **In-app (git-clone & OVA installs):** Settings -> System -> Upgrade. Git-fetches a newer tag, rebuilds, migrates, and restarts — with automatic backup and rollback on failure. CLI equivalent: `sudo scripts/upgrade.sh --to vX.Y.Z`.
- **Offline bundle:** ship the new tarball, then `down` -> `./install.sh --upgrade` -> `up -d`. Preserves `.env` and volumes.
- **Traffic agents:** self-update over WebSocket from the UI (Settings -> Agents -> Build Image, then per-agent Update).

```
# Offline upgrade
cd /opt/packetarch
sudo docker compose down
sudo ./install.sh --upgrade     # loads new images, keeps .env + data
sudo docker compose up -d
```

Backups

The bundle ships backup/restore scripts that snapshot the Postgres DB + PCAP volumes into one tarball. Back up before every upgrade.

```
cd /opt/packetarch
sudo ./packetarch-backup.sh                      # redacts .env secrets
sudo ./packetarch-backup.sh --output /mnt/safe/pa.tgz --with-secrets
sudo ./packetarch-restore.sh /mnt/safe/pa.tgz    # --yes to skip prompt
```

Each tarball holds `postgres.dump`, `pcap_output.tar.gz`, `pcap_uploads.tar.gz`, a redacted/raw `.env`, and a `manifest.json` (version + timestamp).

Uninstall

```
cd /opt/packetarch
sudo docker compose down -v     # -v removes volumes (DATA LOSS)
sudo rm -rf /opt/packetarch
```

Common Operations

Task	Command
Status	`docker compose ps`
Logs	`docker compose logs -f backend`
Restart all	`docker compose restart`
Rebuild backend	`docker compose up -d --build backend`
Stop / start	`docker compose down / docker compose up -d`

Troubleshooting

Two read-only commands answer most of this. Run them before changing anything — between them they name every install failure seen in the field so far.

```
./scripts/check-docker-egress.sh    # can containers build, route and RESOLVE?
./scripts/collect-diagnostics.sh    # one redacted file to send back
```

`collect-diagnostics.sh` locates the install from its own path, so it works on any of the four methods without being told which. It replaces every secret value from `.env` with `<redacted:KEY>` throughout its output, including inside container logs. In the offline bundle and the appliance both scripts sit flat beside `docker-compose.yml`, so drop the `scripts/` prefix.

Which install path am I on?

Layout	Install dir	Tell
Git clone	`wherever you cloned, usually ~/packetarch`	`.git/ is present`
Offline bundle	`/opt/packetarch (--install-dir overrides)`	`VERSION file, no .git/`
Appliance (OVA)	`/opt/packetarch`	`/var/log/packetarch-firstboot.log and .firstboot-done`

Everything is green and the platform is still unreachable

This is the failure that costs the most time, because nothing looks wrong. On a VPN'd corporate laptop the tunnel usually routes `172.16.0.0/12`, which covers almost all of Docker's default address pools (`172.17–172.31.0.0/16`). When the stack's bridge lands inside that block:

- every container starts and `docker compose ps` is green;
- `curl https://localhost/health` *inside* the frontend container returns 200;
- the host still gets a TCP reset on 80/443, because the reply routes out through the VPN instead of back to the bridge;
- and Docker's embedded DNS (`127.0.0.11`) can stop returning answers, so the name `backend` no longer resolves. nginx re-resolves its upstream on every request, so every `/api/` call returns 502 after the resolver timeout and the login page reports **Backend unreachable**.

PacketArch pins its subnet so this cannot happen by accident. If your site routes the default too, pick a free /24:

```
echo 'COMPOSE_SUBNET=10.201.0.0/24' >> .env
docker compose down && docker compose up -d   # a subnet change needs
                                              # the network RECREATED
```

Do not work around this with `extra_hosts` entries in `docker-compose.yml`. They pin container IP addresses that change on every recreate; they cannot help the frontend at all, because nginx resolves through 127.0.0.11 rather than /etc/hosts; and they are a local edit to a TRACKED file, which `scripts/upgrade.sh` refuses to upgrade over (it stops unless you pass `--force`, and then stashes them). Put `COMPOSE_SUBNET` in `.env` instead — .env is untracked, so upgrades leave it alone.

A green healthcheck is a claim, not evidence

Read what a check executes before believing it. `collect-diagnostics.sh` prints the literal text of every one, alongside each container's restart count — a service can report healthy while crashlooping behind it.

Service	Probe	What green proves
postgres	`pg_isready`	the server accepts connections. NOT that credentials work
backend	`curl /health`	the app is up, which needs the database
celery_worker	`celery ping + a real DB connect`	broker and database both reachable
frontend	`curl -fsk https://127.0.0.1/health`	TLS up, 'backend' resolved AND reached (the endpoint is proxied)
host-agent	`heartbeat age + main-loop age`	process alive and its request loop turning

Backend crashlooping on “password authentication failed”

Postgres applies `POSTGRES_PASSWORD` only when it initialises an EMPTY data directory. A data volume that outlived its `.env` keeps the password it was built with, so a regenerated .env silently stops matching. Re-cloning into a fresh directory, deleting .env, or `install.sh --force-env` all produce it. Repair it without losing data:

```
./scripts/fix-db-password.sh       # --check to report only
```

Builds fail on pip / apt / npm / apk

Those steps run inside a bridged build sandbox, not on the host network stack, so a host with perfectly good internet can fail every build. Run the egress preflight: it separates firewall FORWARD drops, a VPN MTU below the bridge's (which HANGS mid-download rather than erroring), an unreachable container resolver, and subnet collisions. Prefer the `/etc/docker/daemon.json` fix — most of those causes also break RUNTIME egress to a Cyber Vision Center, a CML server or an AI provider, and a build that succeeds is not an install that works. The supported stopgap is an .env line, never a compose edit:

```
echo 'DOCKER_BUILD_NETWORK=host' >> .env
docker compose up -d --build
```

PacketArch is © 2026 Rocky Smith (rocky.d.smith@proton.me) and is licensed under GPL-3.0. Issues & questions: github.com/ip-aegis/PacketArch/issues